A CASE-STUDY BASED REPORT

ON

Predicting Indian Premier League Match Outcomes Using Machine Learning on Historical Match and Ball-by-Ball Data

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

A.Y.: 2026-27

by

Bhanu Jakka	**2520030006**
V. Charishma	**2520030140**

Under the Esteemed guidance of

Mr. Shaik Asif

Asst.Professor, Dept. of CSE

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract **4**

Introduction **5**

Background 5
Motivation for the Project 5
Overview of the Work Carried Out 5
Scope and Responsible-Use Statement 6

Problem Statement **7**

Case-Study Scenario 7
Central Case-Study Question 8
Formal Problem Definition 8
Objectives that Guide the Study 8
Assumptions and Limitations 9

Dataset and Exploratory Data Analysis **10**

About the Dataset 10
Purpose of EDA 11
Preparation Steps Actually Performed 11
EDA Exploration - Implementation 12
Summary of EDA Findings 17

Algorithm Selection and Implementation **18**

Conclusion **23**

Signature of the Team members

Abstract

The Indian Premier League (IPL) is one of the most closely contested franchise cricket tournaments in the world, and forecasting the winner of a match before it begins is a popular but difficult analytics problem. This case study examines how much of that outcome can be predicted from publicly available historical data using classical machine learning. Two tables were used: a ball-by-ball file of 260,920 deliveries and a match-level file of 1,095 matches, running from the inaugural 2008 match to the 2024 final.

Exploratory data analysis of the delivery data established the scoring rhythm of T20 cricket (about 8.0 runs per over, with scoring dipping once the powerplay ends and peaking in the final overs), the dominance of catches among dismissals (about 62%), and the quality of the data (no duplicate rows; the only missing values are fields that are empty by design). Nineteen pre-match features were then engineered for the 1,090 matches that produced a result, each computed only from matches played earlier. The data were split 70/15/15 with stratification, and Logistic Regression and Random Forest classifiers were trained and compared.

On the held-out test set, the models using all 19 features reached 51.2% (Logistic Regression) and 50.6% (Random Forest) accuracy. After four weak features were removed, Logistic Regression reached 53.7% accuracy (F1 0.553, ROC-AUC 0.522) and Random Forest 53.0% (F1 0.550, ROC-AUC 0.546). Because always predicting a win for the first-listed team scores 51.2% on the same test set, the improvement is small and lies within sampling noise. The study concludes that team-level historical form carries very little predictive signal for IPL results, and it outlines the data and methods that would be needed to do better.

Keywords: IPL, cricket analytics, match outcome prediction, feature engineering, Logistic Regression, Random Forest.

Introduction

Background

The Indian Premier League, launched in 2008, is a franchise Twenty20 (T20) cricket league in which each side bats for a maximum of 20 overs. Matches last about three hours, results are often decided by small margins, and a single over, a dropped catch or a toss decision can change the outcome. Over seventeen seasons the league has produced a rich public record: every delivery is logged with its batter, bowler, runs, extras and dismissal details, and every match with its teams, venue, toss and result.

Machine learning treats such records as examples from which patterns can be learned. Predicting the winner of a match is a binary classification problem: given what is known about two teams before the game, output which of them is more likely to win. It is a useful teaching problem because the data are real, the target is unambiguous and, as this report shows, the problem is genuinely hard.

Motivation for the Project

Three motivations drove this project.

- **Practical interest.** Pre-match win probabilities are used by broadcasters, analysts and fantasy-sports players, and a transparent, reproducible baseline is a sensible starting point for any such system.
- **Learning value.** The problem exercises the complete machine-learning workflow: data understanding, cleaning, feature engineering, splitting, model training, evaluation and diagnosis, on a dataset large enough to be realistic.
- **A genuine scientific question.** How much of a T20 result is explained by team-level history, and how much is left to chance and in-match events? An honest answer, even a negative one, is informative.

Overview of the Work Carried Out

The work was organised as a sequence of five notebooks, each producing artefacts consumed by the next:

1. **Exploratory data analysis** (EDA.ipynb): inspected the 260,920-row delivery file for structure, missing values and duplicates, and summarised scoring, dismissals, top performers and run rates with seven charts.

2. **Feature engineering** (Feature_Engineering_IPL_MatchPrediction.ipynb): standardised franchise names, sorted matches by date, removed matches without a winner, and built 19 pre-match features and the target label.
3. **Train, validation and test split** (Train_Validation_Test_Split_IPL_MatchPrediction.ipynb): created a stratified 70/15/15 split with a fixed seed and verified the class balance.
4. **Model selection and training** (Model_Selection_Training_IPL_MatchPrediction.ipynb): trained Logistic Regression and Random Forest on all 19 features, compared training and validation accuracy, and retrained both on a reduced 15-feature set.
5. **Model evaluation** (Model_Evaluation_IPL_MatchPrediction.ipynb): scored the models on the test set (accuracy, precision, recall, F1, ROC-AUC) and ran diagnostics (feature spread, correlations, confusion matrices, ROC curves, feature importance).

Scope and Responsible-Use Statement

Scope. The study predicts, for each completed IPL match from 2008 to 2024, whether the team listed first (team1) wins, using only information available before the match: the teams' recent record, head-to-head and venue history, home advantage, toss outcome and rolling batting and bowling averages. Two classical algorithms are compared. Player-level data, live prediction and deep learning are out of scope.

Responsible use. The following principles were applied and should be respected by anyone reusing this work:

- The models are **not intended for betting or gambling** and must not be presented as reliable forecasts. Their accuracy is close to chance, as documented below.
- All results, including the disappointing ones, are reported in full. No metric was selected after the fact to make the outcome look better, and limitations are stated openly in the sections that follow.
- The data are public sports records; no private or sensitive personal information is used, and modelling is at team level.
- The records span several eras and franchise rebrandings, so conclusions should not be generalised to other leagues or seasons without re-validation.

Problem Statement

Case-Study Scenario

Background

It is the eve of an Indian Premier League (IPL) match. Rohan, a sports analyst working for a broadcasting network, has been asked a simple but high-stakes question by his producer: *"Who is going to win tomorrow's match, and can we support that prediction with data rather than opinion?"*

Rohan has access to seventeen years of IPL history, covering every match played since 2008, along with ball-by-ball records of over 260,000 individual deliveries. He knows that pundits and fans typically rely on instinct: a team's reputation, a star player's current form, or which side simply "feels" stronger on the day. Rohan wants something more rigorous. A team's recent performance, its history against the specific opponent, whether it is playing at home or away, and even the outcome of the toss could all be quietly influencing the result, and no analyst can consistently weigh all of these factors by intuition alone across hundreds of matches.

The Approach

Rohan turns the producer's question into a machine learning problem. He treats "Will Team A beat Team B tomorrow?" as a binary classification task, where the outcome is simply win or loss for a chosen team. Using the historical dataset, he engineers features that reflect how an analyst would naturally reason about a match: each team's recent win rate, their head-to-head record against tomorrow's opponent, whether they are playing at their home venue, who won the toss and what decision they made, and each team's average batting and bowling performance in recent matches.

He then builds two predictive models to answer the producer's question. The first, **Logistic Regression**, provides a transparent, interpretable answer, allowing Rohan to identify exactly which factors are pushing a prediction toward a win or a loss, which matters when he must justify a claim on air. The second, **Random Forest**, is less easily interpreted but better suited to capturing complex, non-obvious combinations of factors, such as a team having a home advantage that only becomes meaningful when combined with winning the toss.

The Outcome

After training both models on past matches and testing them on matches they have never seen, Rohan compares their predictions against actual results. Both models perform only modestly better than random chance, a humbling but honest outcome. On closer analysis, he finds that a

team's recent scoring and wicket-taking form carries more predictive weight than the toss or home advantage, a useful insight for his producer even though a confident, high-accuracy prediction remains difficult to achieve.

Relevance to the Project

This scenario reflects the real-world motivation behind the project: cricket, like many sports, resists easy prediction, and the value of a machine learning approach lies not in producing a perfect forecast, but in offering a systematic, data-driven way to reason about probability where human intuition alone is insufficient. The project's use of Logistic Regression and Random Forest, applied to engineered features such as recent form, head-to-head history, venue advantage, and toss outcome, is a direct implementation of the approach Rohan follows in this scenario.

Central Case-Study Question

Rohan's producer asks who will win tomorrow's match and whether the answer can rest on data rather than opinion. Using only information available before a match begins, can historical IPL data support a classifier that predicts the winner more accurately than chance, and which pre-match factors contribute most to that prediction?

Formal Problem Definition

Let the completed matches be ordered by date, m = 1, …, 1090. For match m, the two teams are team1 and team2 as listed in the match file, played at venue v on date d. A feature vector x(m) of 19 numeric values is computed from matches with a date strictly earlier than d, together with the toss outcome and a home-city indicator. The target is y(m) = 1 if team1 wins and y(m) = 0 otherwise. The task is to learn a function f that maps x(m) to a predicted label and to measure how well it generalises to unseen matches.

- **Learning task:** supervised binary classification.
- **Instances:** 1,090 IPL matches with a recorded winner (1,095 in the raw file).
- **Inputs and output:** 19 numeric pre-match features (Table 4); the target team1_win is 1 for 50.9% of matches.
- **Evaluation:** accuracy, precision, recall, F1 score and ROC-AUC on a held-out test set, compared against the baseline of always predicting that team1 wins (51.2% on the test set).

Objectives that Guide the Study

1. Understand the structure, quality and scoring patterns of the delivery data through EDA.

2. Clean the match data so that team identities are consistent across seasons.
3. Engineer pre-match features that use no information from the future.
4. Split the data into training, validation and test sets that preserve the class balance.
5. Train and compare an interpretable linear model and a non-linear ensemble model.
6. Evaluate against a naive baseline, diagnose the limits of performance and identify key features.

Assumptions and Limitations

Assumptions

- A franchise's recent results, scoring and bowling form over its last five matches are informative about its next match.
- Renamed franchises are treated as continuous teams (Delhi Daredevils to Delhi Capitals, Kings XI Punjab to Punjab Kings, Deccan Chargers to Sunrisers Hyderabad); the last mapping is a simplification, as these were different franchises. Other spelling variants (Royal Challengers Bangalore/Bengaluru, Rising Pune Supergiant/Supergiants) were not merged.
- A team's home city is the city in which it has played most often, and matches with no recorded winner are excluded rather than modelled as a third class.

Limitations

- **Small sample and limited information.** Only 1,090 matches are available and the test set holds 164, so the 95% margin of error on an accuracy of about 53% is roughly plus or minus 7.6 percentage points; no player, pitch, weather or team-selection data are included.
- **Evaluation design.** The split is random rather than chronological, so test matches come from the same eras as training matches, and there is no cross-validation or hyperparameter tuning.
- **Feature reduction used test-set information.** Dropping four features followed diagnostics that included the test split, so the reduced-model scores are slightly optimistic.
- **Minor leakage and ordering.** Home-city assignment and fallback defaults use the whole dataset, and team1 is far more often the home side (50% against 11% for team2 in the test set).

Dataset and Exploratory Data Analysis

About the Dataset

The study uses two related tables from a publicly available IPL dataset. The first (deliveries.csv) records every legal and illegal delivery bowled; the second (matches.csv) records one row per match. Both files begin with the inaugural 2008 fixture and end with the 2024 final, and they are linked by the match identifier (match_id in the delivery file, id in the match file).

Table 1: *Overview of the two data files*

Characteristic	deliveries.csv	matches.csv
Grain	One row per delivery	One row per match
Rows	260,920	1,095 (1,090 with a winner)
Columns	17 (8 integer, 9 text)	20
Memory footprint	about 33.8 MB	small
Role in this study	EDA; per-match team runs, runs conceded and wickets	Teams, date, venue, city, toss and winner; target label

The 17 columns of the delivery file are described in Table 2. The match file supplies the fields used for feature engineering: id, date, team1, team2, toss_winner, toss_decision, winner, city and venue. Its remaining columns were not used.

Table 2: *Columns of deliveries.csv*

Column	Type	Meaning
match_id	int	Identifier linking the delivery to its match
inning	int	Innings number (1 or 2; up to 6 where super overs are included)
batting_team, bowling_team	text	Franchise batting and bowling in this innings
over, ball	int	Over number (0 to 19) and ball within the over
batter, bowler, non_striker	text	Players involved in the delivery
batsman_runs	int	Runs scored off the bat (0 to 6)
extra_runs, extras_type	int, text	Runs from extras (wides, no-balls, byes, leg-byes) and their type
total_runs	int	batsman_runs + extra_runs
is_wicket	int	1 if a dismissal occurred on this delivery
player_dismissed, dismissal_kind, fielder	text	Details of the dismissal, where there was one

Data quality

No duplicate rows were found. The only missing values are in the four columns shown in Table 3, and in each case the gaps are structural rather than errors: an extras type exists only when extras were conceded, dismissal details exist only when a wicket fell, and a fielder is recorded only for

catches, run-outs and stumpings. The count of non-null dismissal_kind values (12,950) equals the sum of is_wicket, which confirms the fields are consistent with one another. No imputation was therefore required.

Table 3: *Missing values in deliveries.csv*

Column	Missing	Percentage	Reason
fielder	251,566	96.41%	Only catches, run-outs and stumpings involve a fielder
dismissal_kind	247,970	95.04%	Empty when no wicket fell
player_dismissed	247,970	95.04%	Empty when no wicket fell
extras_type	246,795	94.59%	Empty when no extras were conceded

Purpose of EDA

Exploratory data analysis was carried out before any modelling to answer practical questions that shape later decisions:

- Is the data complete, consistent and free of duplicates, and what do the missing values mean?
- What does a typical delivery, over and innings look like, so that engineered features such as average runs scored and wickets taken can be sanity-checked?
- Do team names remain consistent across seasons, and what would happen if they did not?
- Which teams, batters and bowlers dominate, and is there a visible structure in how runs and wickets are distributed through an innings?
- Which numeric variables are related, and is there any early indication of what a model could exploit?

Preparation Steps Actually Performed

The following preparation steps were carried out in the notebooks, in this order:

1. **Standardised team names** in team1, team2, toss_winner, winner and in both team columns of the delivery file (Delhi Daredevils to Delhi Capitals, Deccan Chargers to Sunrisers Hyderabad, Kings XI Punjab to Punjab Kings).
2. **Sorted matches chronologically** by date so that every rolling feature looks only backwards in time.
3. **Removed matches without a winner.** The match file shrank from 1,095 to 1,090 rows.
4. **Aggregated delivery data to team-per-match statistics:** runs scored, runs conceded and wickets taken by each team in each match.

5. **Computed pre-match features** (Table 4) for every match using only earlier matches. Where a team had no history, neutral defaults were used: 0.5 for form, head-to-head and venue ratios, 14 days for rest, and the overall average for batting and bowling statistics.
6. **Created the target** team1_win (1 if team1 won, 0 otherwise) and dropped any rows with missing values. No rows were lost at this step, leaving 1,090 rows and 20 columns (19 features plus the target).
7. **Split the data** 70/15/15 into training (762), validation (164) and test (164) sets, stratified on the target with a fixed random seed of 42.
8. **Scaled the features** with StandardScaler fitted on the training data only. Scaling was applied for Logistic Regression and not for the tree-based Random Forest.

Table 4: *The 19 engineered features (each exists for team1 and team2 unless noted)*

Feature	Description
form	Weighted share of wins in the last five matches; recent matches get higher weights (0.5 to 1.5)
win_streak	Number of consecutive wins immediately before the match
h2h_team1_win_ratio (team1 only)	Share of earlier meetings between the two teams that team1 won
home	1 if the match city is the team's most frequent home city
toss_won_by_team1 (team1 only)	1 if team1 won the toss
toss_decision_bat (shared)	1 if the toss winner chose to bat first
venue_form	Share of the team's earlier matches won at this venue
avg_runs_scored	Mean runs scored in the team's last five matches
avg_runs_conceded	Mean runs conceded in the last five matches
avg_wickets_taken	Mean wickets taken in the last five matches
rest_days	Days since the team's previous match

EDA Exploration - Implementation

This section presents the analysis performed on the 260,920 deliveries, with each result followed by its interpretation.

Overall totals

The tournament data contain 347,756 runs, of which 330,064 (94.9%) came off the bat and 17,692 (5.1%) were extras. There were 12,950 dismissals, which is one wicket every 20 deliveries (4.96% of balls). The mean total per delivery is 1.33 runs, equivalent to about 8.0 runs per over, which is the benchmark scoring rate in this dataset.

Runs by batting team

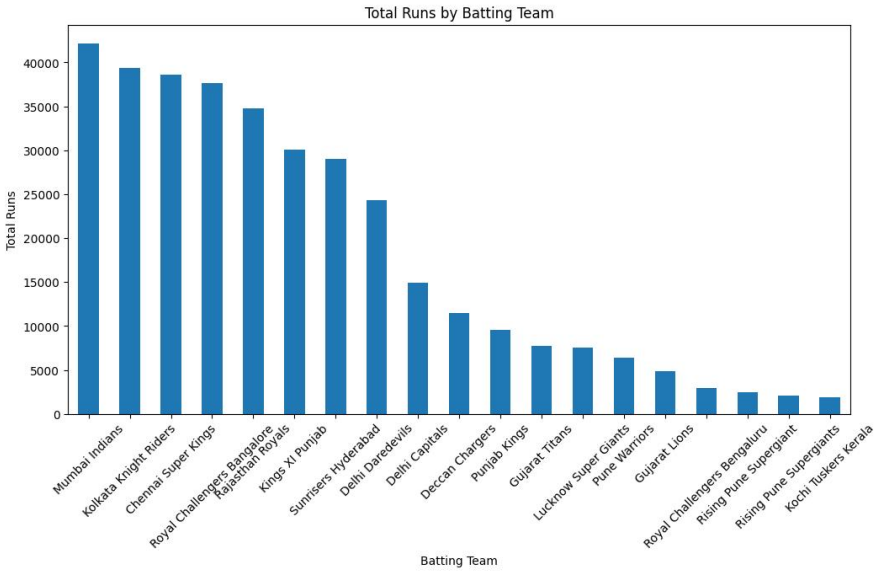


Figure 1: *Total runs scored by each batting team*

Mumbai Indians lead with 42,176 runs, followed by Kolkata Knight Riders (39,331), Chennai Super Kings (38,629) and Royal Challengers Bangalore (37,692). Two caveats matter for modelling. First, totals depend on how many matches a franchise has played, so they are not a measure of strength. Second, the chart exposes the team-name problem: Delhi appears twice (24,296 as Daredevils and 14,900 as Capitals, 39,196 combined) and so does Punjab (30,064 and 9,536, 39,600 combined). Merging the variants moves both franchises close to the top four. This is why name standardisation was the first preparation step.

Top run scorers

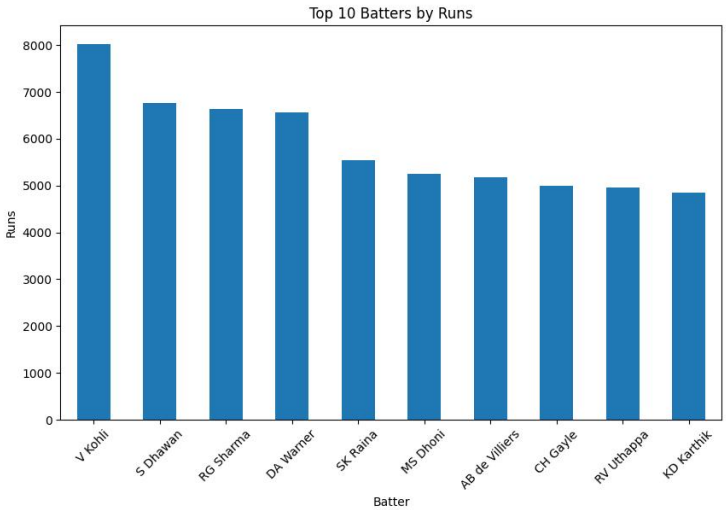


Figure 2: *Ten leading run scorers*

Virat Kohli is clearly ahead with 8,014 runs, followed by Shikhar Dhawan (6,769), Rohit Sharma (6,630), David Warner (6,567) and Suresh Raina (5,536). The leaders have all played for many seasons, so cumulative runs reward longevity as much as quality. Individual player strength is not part of the team-level feature set, which is one reason the models struggle.

Wickets and dismissal types

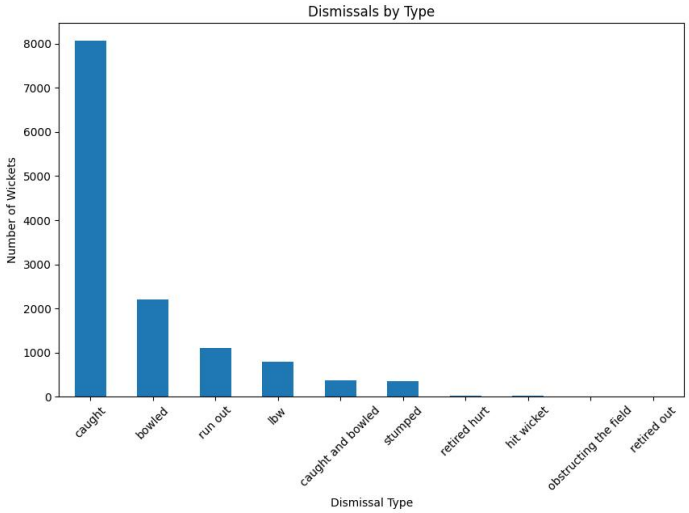


Figure 3: *Number of wickets by dismissal type*

Catches account for 8,063 of the 12,950 dismissals (62.3%), followed by bowled (2,212; 17.1%), run out (1,114; 8.6%), lbw (800; 6.2%), caught and bowled (367; 2.8%) and stumped (358; 2.8%). The remaining 36 are rare events such as retired hurt, hit wicket, obstructing the field and retired out. The strong dominance of caught dismissals reflects the aggressive, boundary-hunting nature of T20 batting.

Leading wicket takers

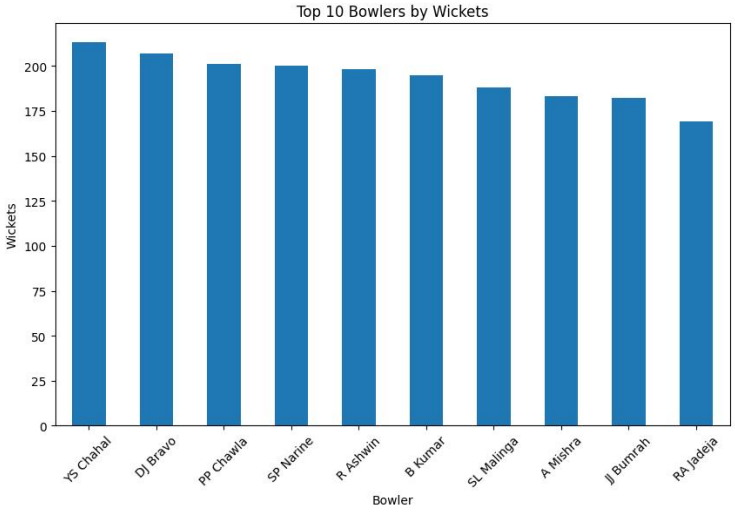


Figure 4: *Ten leading bowlers by dismissals*

Yuzvendra Chahal (213), Dwayne Bravo (207), Piyush Chawla (201), Sunil Narine (200) and R Ashwin (198) top the list. The count is derived from the is_wicket flag, so it includes run-outs and retirements that official bowling figures do not credit to the bowler. Spin bowlers feature prominently.

Runs per delivery

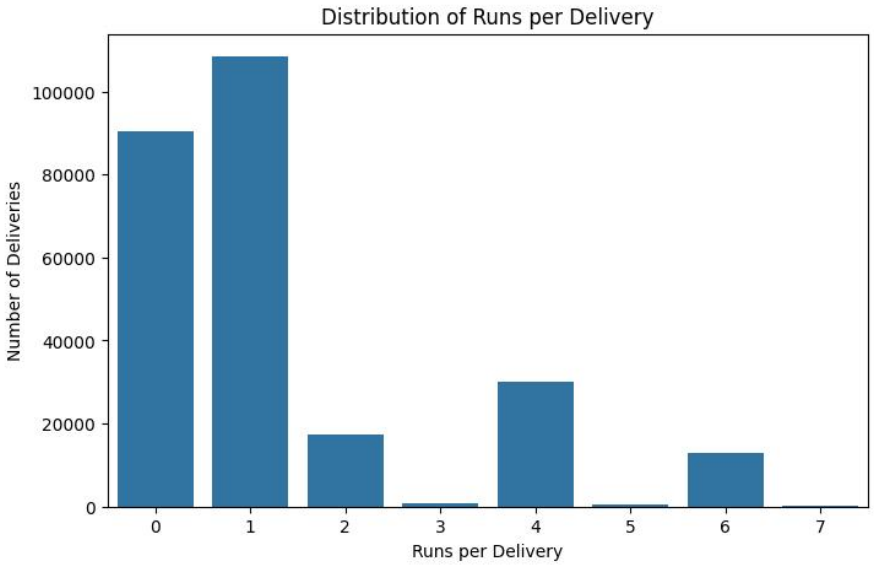


Figure 5: *Distribution of runs per delivery*

The distribution is dominated by dot balls (roughly 90,000) and singles (roughly 108,000), which together make up about three quarters of all deliveries. Fours (about 30,000) and sixes (about 13,000) are far rarer but contribute a large share of the runs. Values of 3, 5 and 7 are uncommon. The shape is right-skewed, which explains why a few boundary overs can swing a match.

Scoring through an innings

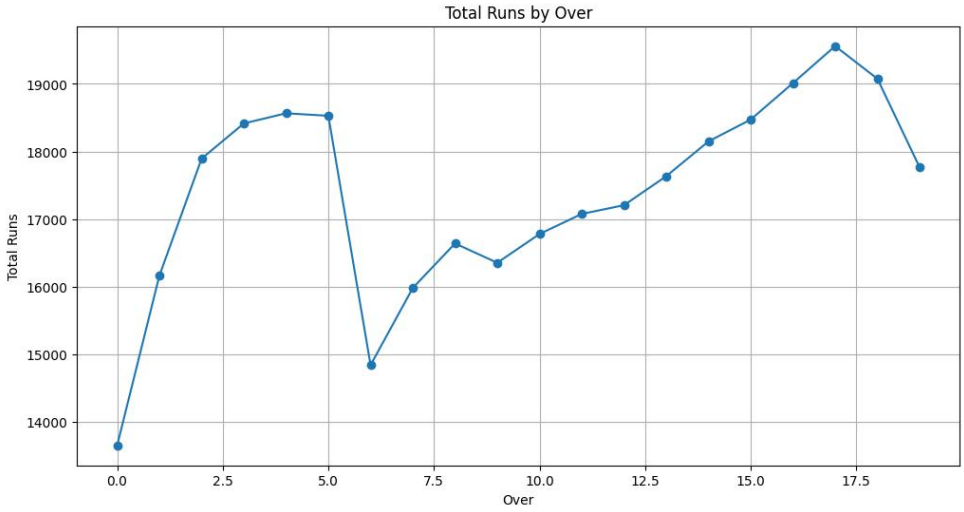


Figure 6: *Total runs scored in each over (over 0 is the first over)*

The first over is the lowest-scoring (13,651 runs in total), after which scoring rises to about 18,500 runs in each of overs 4 to 6 of the powerplay (displayed as overs 3 to 5 on the chart's zero-based axis). It then drops to 14,839 in the seventh over, when fielding restrictions are lifted, and climbs steadily through the middle overs to a peak of 19,559 in the eighteenth over (axis value 17). The nineteenth over is also high (19,076) before the final over eases to 17,765. The innings therefore has a clear three-phase structure: powerplay, consolidation and acceleration.

Correlation analysis

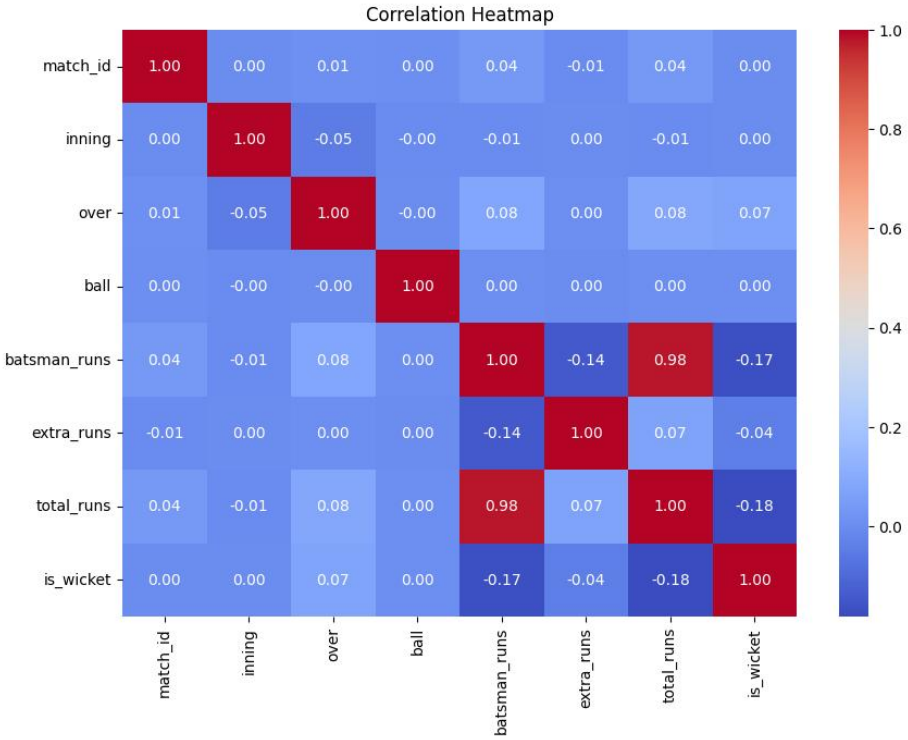


Figure 7: *Correlation heatmap of the numeric delivery columns*

The strongest relationship is between batsman_runs and total_runs (0.98), which is expected because total runs are batsman runs plus extras. Wickets are negatively related to runs on the same ball (-0.18 with total_runs and -0.17 with batsman_runs). The over number is mildly positively related to both runs (0.08) and wickets (0.07): late overs see more scoring and more dismissals. Identifier-style columns such as match_id and inning show almost no correlation with anything. These are descriptive relationships at the delivery level and they do not directly predict match winners, which is why match-level features had to be engineered.

Relationship between engineered features and the target

After feature engineering, the correlation of each feature with the target was checked (Table 5). Every coefficient is very small: the largest in absolute value is 0.073 (team1_home) and the smallest 0.0006. Features with the expected sign appear (home advantage helps, conceding more

runs hurts), but so do counterintuitive ones, such as a longer win streak for team2 being slightly positively related to a team1 win. This pattern is typical of noise and is the first sign that predictive power will be limited.

Table 5: *Correlation of each feature with team1_win (all 1,090 matches)*

Feature	Correlation	Feature	Correlation
team1_home	+0.0730	team2_avg_wickets_taken	+0.0071
team2_win_streak	+0.0613	team1_venue_form	+0.0007
toss_decision_bat	+0.0590	team2_avg_runs_scored	+0.0006
h2h_team1_win_ratio	+0.0378	team1_win_streak	-0.0059
team1_form	+0.0375	team2_rest_days	-0.0097
team2_form	+0.0367	team2_avg_runs_conceded	-0.0158
team1_avg_wickets_taken	+0.0273	team1_rest_days	-0.0162
team1_avg_runs_scored	+0.0228	team1_avg_runs_conceded	-0.0452
toss_won_by_team1	+0.0212	team2_home	-0.0718
team2_venue_form	+0.0082		

Summary of EDA Findings

- **Clean and complete.** There are no duplicate rows and the missing values are structural, so no imputation was necessary. The delivery table has 260,920 rows and 17 columns.
- **Scoring benchmark.** About 8.0 runs per over, 1.33 runs per ball, and a wicket roughly every 20 balls.
- **Three-phase innings.** Runs per over dip when the powerplay ends and peak around the 18th over, so averages based on whole-innings totals hide important structure.
- **Dismissals.** Catches make up about 62% of wickets, followed by bowled (17%) and run out (9%).
- **Team names are inconsistent.** Franchise renames split totals across labels, which justified the name standardisation applied before feature engineering.
- **Individual stars matter but are not captured.** The leading batters and bowlers are long-serving players, yet the team-level feature set does not represent them.
- **Weak signals.** All 19 engineered features correlate with the match result at |r| of 0.073 or less, which anticipated the modest model performance.

Algorithm Selection and Implementation

Choice of learning task and algorithms

Predicting whether team1 wins is a supervised binary classification problem with 19 numeric inputs and about 760 training examples. For a small tabular dataset of this kind, two complementary algorithms were selected:

- **Logistic Regression** is a linear model that outputs a probability through the logistic function. It is fast, hard to overfit on small data and easy to interpret, which makes it a strong baseline. It was trained on standardised features with max_iter set to 1000 and random_state 42.
- **Random Forest** is an ensemble of decision trees that can capture non-linear effects and interactions, such as a team doing well specifically at a given venue, and is insensitive to feature scale. It was configured with 200 trees, a maximum depth of 8 and random_state 42, and was trained on unscaled features.

Comparing a simple linear model with a flexible ensemble makes it possible to tell whether any extra structure exists in the data beyond what a linear combination of features captures. More powerful methods, such as gradient boosting, were not tried; they are suggested as future work.

Implementation workflow

The workflow followed the standard supervised-learning protocol. The 1,090 rows were split with stratification into 762 training, 164 validation and 164 test examples, the target proportions being 50.9%, 50.6% and 51.2% for team1 wins in the three subsets. Models were fitted on the training set, compared on the validation set, and the test set was reserved for the final assessment. The scaler, Logistic Regression and Random Forest were serialised with joblib so that the evaluation notebook could reload exactly the fitted objects.

Training and validation results (all 19 features)

Table 6: *Training and validation accuracy with all 19 features*

Model	Train accuracy	Validation accuracy
Logistic Regression	57.1%	51.2%
Random Forest	97.9%	52.4%

The Random Forest fits the training data almost perfectly (97.9%) yet reaches only 52.4% on validation, a gap of more than 45 points. This is a textbook case of overfitting: with so little signal in the features, the trees memorise individual matches. Logistic Regression is far more stable (57.1% against 51.2%) but is also barely above chance on unseen data.

Test-set evaluation (all 19 features)

Table 7: *Test-set metrics with all 19 features (164 matches)*

Model	Accuracy	Precision	Recall	F1 score	ROC-AUC
Logistic Regression	0.512	0.523	0.548	0.535	0.514
Random Forest	0.506	0.518	0.524	0.521	0.501

Both models perform at chance level. Logistic Regression's accuracy of 51.2% is identical to the proportion of team1 wins in the test set (84 of 164), and the Random Forest's ROC-AUC of 0.501 is indistinguishable from random guessing (0.5). Logistic Regression was marginally better on every metric.

Diagnosing the weak performance

Two diagnostics were run on the engineered data. A feature-spread check showed that the features have sensible ranges, with one exception: rest days had a mean of about 19 days but a standard deviation of about 71, because long gaps between seasons dominate the variable and make it a poor measure of fatigue. A feature-target correlation check (Table 5) showed that no feature correlates with the outcome above $|r| = 0.073$. Together these indicated that several features were noisy, and that the overfitting seen in the Random Forest was largely the result of fitting that noise.

Feature reduction and retraining

Four features were removed: team1_rest_days, team2_rest_days, team1_win_streak and team2_win_streak. Rest days are dominated by off-season gaps, and win streaks largely duplicate the information already in the weighted form feature. Both models were then retrained on the remaining 15 features with the same split strategy and hyperparameters, and evaluated on the test set.

Table 8: *Test-set metrics after reducing to 15 features (164 matches)*

Model	Accuracy	Precision	Recall	F1 score	ROC-AUC
Logistic Regression (reduced)	0.537	0.547	0.560	0.553	0.522
Random Forest (reduced)	0.530	0.540	0.560	0.550	0.546

Both models improved: accuracy rose by about 2.4 percentage points for each model, and the Random Forest's ROC-AUC increased from 0.501 to 0.546. The class-wise results are similarly balanced. For Logistic Regression, precision and recall are 0.53 and 0.51 for losses and 0.55 and 0.56 for wins; for Random Forest they are 0.52 and 0.50 for losses and 0.54 and 0.56 for wins.

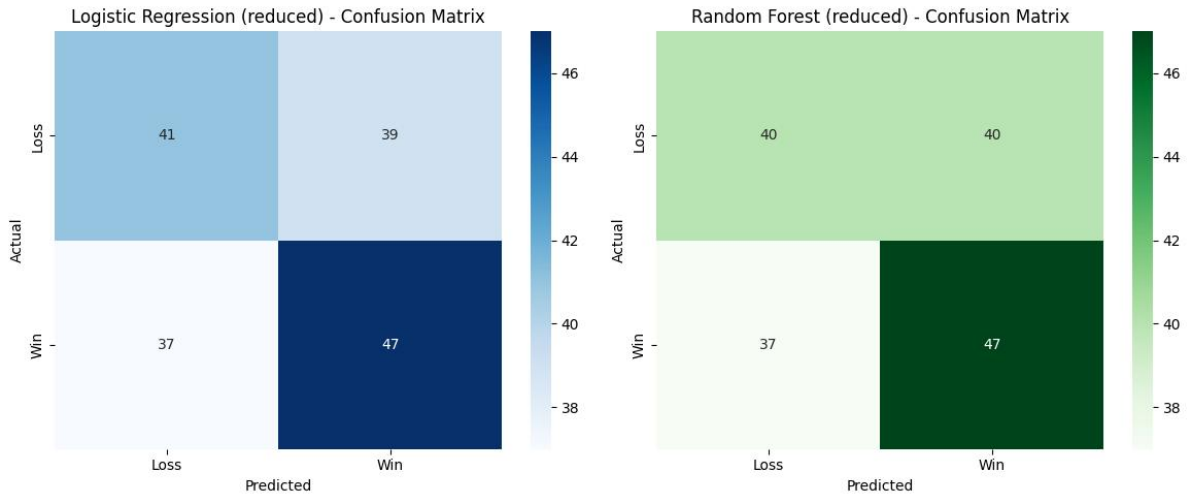


Figure 8: *Confusion matrices on the test set (reduced feature set)*

The confusion matrices show that Logistic Regression classified 41 losses and 47 wins correctly (88 of 164) while Random Forest classified 40 and 47 correctly (87 of 164). Both models make nearly as many false positives (39 and 40) as false negatives (37), so there is no systematic bias towards predicting either outcome.

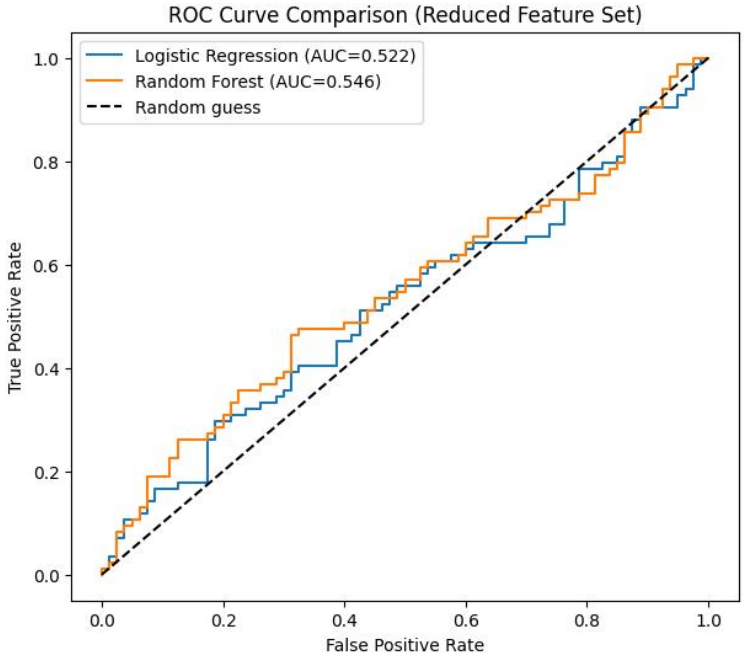


Figure 9: *ROC curves of the reduced models against random guessing*

The ROC curves hug the diagonal. The areas under the curve are 0.522 for Logistic Regression and 0.546 for Random Forest. Random Forest ranks matches slightly better by probability even though its hard predictions are no more accurate.

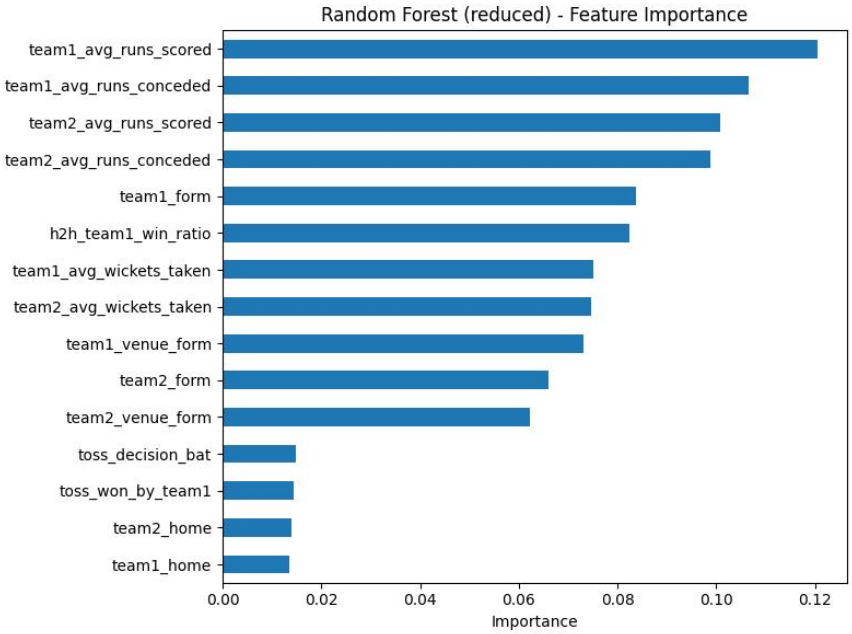


Figure 10: *Random Forest feature importance (reduced feature set)*

The most important features are the rolling batting and bowling averages: team1 runs scored (0.120), team1 runs conceded (0.106), team2 runs scored (0.101) and team2 runs conceded (0.099). Team form, head-to-head ratio, wickets taken and venue form follow at 0.06 to 0.08 each, while the toss and home indicators are least important (about 0.014 each). The importances are fairly flat, meaning no single factor dominates. Continuous features also tend to receive higher importance in tree models simply because they offer more split points, so this ranking should be read cautiously.

Model comparison and selection

Table 9: *Summary of all four model configurations on the test set*

Configuration	Accuracy	F1 score	ROC-AUC
Baseline: always predict team1 wins	0.512	-	0.500
Logistic Regression, 19 features	0.512	0.535	0.514
Random Forest, 19 features	0.506	0.521	0.501
Logistic Regression, 15 features	0.537	0.553	0.522
Random Forest, 15 features	0.530	0.550	0.546

Logistic Regression with 15 features gives the highest accuracy and F1 score, and Random Forest with 15 features the highest ROC-AUC. The differences between them are tiny: one match out of 164 separates their accuracies (88 against 87 correct predictions), and the best model gets only four more matches right than the naive baseline of 84. Logistic Regression with the reduced feature set is therefore recommended as the preferred model on grounds of simplicity,

stability and interpretability, but the evidence does not support claiming that any model is reliably better than guessing. Both final models and the final scaler were saved for reuse.

Conclusion

This case study set out to determine whether public historical IPL data could support a pre-match winner predictor that beats chance. A full machine-learning pipeline was built and documented: exploratory analysis of 260,920 deliveries, leakage-aware engineering of 19 pre-match features for 1,090 matches, a stratified 70/15/15 split, and the training and evaluation of Logistic Regression and Random Forest classifiers.

The analysis produced several clear findings. The delivery data are clean and reveal the characteristic shape of a T20 innings, with about 8 runs per over, a dip after the powerplay and a surge in the final overs. Franchise renames must be harmonised before team-level statistics are meaningful. Most importantly, the engineered features carry almost no linear signal about the result (|r| below 0.08), the Random Forest overfits badly when given all 19 features (97.9% training against 52.4% validation accuracy), and the best test accuracy after feature reduction is only 53.7% with ROC-AUC of 0.522 to 0.546, against a baseline of 51.2%. With 164 test matches, such a difference cannot be distinguished from chance.

The answer to the central question is therefore cautious: team-level historical form, head-to-head record, venue history, home advantage and toss alone explain very little of who wins an IPL match. This is a meaningful result in itself: it is consistent with the parity of the league and the large role of in-match events, and it argues against presenting any such model as a trustworthy forecaster.

Future work

- Use a **chronological split** with rolling-origin cross-validation, and tune hyperparameters on validation folds only, after unifying all team-name variants.
- Add **player-level features** such as the strength of the announced eleven, recent individual form and bowler-batter match-ups, and context such as pitch type, dew and weather.
- Use stronger team-strength summaries such as **Elo or Bayesian ratings**, and try gradient boosting models and calibrated probabilities.
- Exploit the ball-by-ball data for **live win-probability** modelling, where the in-match state (score, wickets, overs left, target) is far more informative than pre-match history.

In summary, the project built and critically evaluated an end-to-end prediction pipeline, and shows that disciplined evaluation can matter more than a headline accuracy figure.